



Ginásio - Relatório

Beatriz Santos - 50473
Inês Santos - 49436
Manoela Azevedo - 50034

Trabalho para a Unidade Curricular
Programação Orientada a Objetos
1º ciclo de estudos em Informática Web, Móvel e na Nuvem

Docente: Paul Andrew Crocker

Dezembro de 2023

Índice

Objetivo principal.....	3
Estrutura do Projeto.....	3
Funcionalidades Adicionais.....	3
Classes.....	4
Main().....	4
Interface Gráfica.....	4
Funcionalidades Principais.....	4
Considerações Adicionais.....	4
Ficheiro().....	5
Constante.....	5
Método inserirAula().....	5
Método consultarAula().....	5
Método eliminarAula().....	5
Método verAula().....	5
Manipulação de Exceções.....	5
Horario().....	6
Atributo.....	6
Construtor.....	6
Método adicionarAula.....	6
Método removerAula.....	6
Método imprimirHorario.....	6
Aula().....	7
Atributos.....	7
Construtor.....	7
Métodos Acessores.....	7
Métodos de Exibição.....	7
Pessoa().....	8
Atributos.....	8
Construtores.....	8
Métodos Acessores.....	8
Método toString().....	8
Aluno().....	8
Atributos.....	9
Construtores.....	9
Métodos Acessores.....	9
Método toString().....	9
Trainer().....	9
Atributos.....	9
Construtor.....	9
Métodos Acessores.....	10
Método toString().....	10

Objetivo principal

O principal propósito do projeto consiste em oferecer aos utilizadores uma experiência descomplicada e eficaz no agendamento de aulas de ginásio. Através de uma interface intuitiva, o utilizador pode inserir detalhes cruciais, como o dia da semana, a hora e o nome da aula. Essa abordagem simplificada visa tornar o processo de criação de um horário de aulas acessível e eficiente.

Estrutura do Projeto

O projeto é composto por cinco classes principais: "Main", "Horario", "Aula", "Pessoa" e "Ficheiro". Cada classe desempenha um papel específico na recolha e organização dos dados do utilizador. Adicionalmente, foram implementadas duas subclasses derivadas da classe "Pessoa": "Trainer" e "Aluno", proporcionando uma identificação clara e distinção entre treinadores e alunos.

Funcionalidades Adicionais

Além do agendamento de aulas individuais, o projeto incorpora a flexibilidade para aulas em grupo, proporcionando uma experiência abrangente no contexto de um ginásio. Essa funcionalidade adicional visa atender às diversas necessidades dos utilizadores, oferecendo opções tanto para aulas personalizadas quanto para sessões em grupo.

No geral, a estrutura do projeto foi concebida para oferecer uma solução completa e intuitiva, promovendo a organização do tempo e a visualização clara das atividades planeadas. O uso de subclasses específicas para treinadores e alunos aprimora a capacidade do sistema de identificar e distinguir entre diferentes tipos de utilizadores, contribuindo para uma experiência mais personalizada.

Classes

Main()

Interface Gráfica

- Utiliza a biblioteca Swing para criar uma interface gráfica simples.
- Botões na interface: "Ver Horário", "Adicionar Dados", "Consultar Aulas" e "Eliminar Aulas".

Funcionalidades Principais

- Ver Horário: Ao clicar no botão "Ver Horário" o horário é exibido na saída padrão.
- Adicionar Dados: Ao clicar no botão "Adicionar Dados", o utilizador é solicitado a inserir informações sobre a aula, incluindo dia, hora, nome do trainer e participantes. Os dados do trainer e dos participantes são obtidos através da consola. As informações são utilizadas para criar instâncias de Trainer, Aluno, e Aula. A aula é adicionada ao horário. Os dados do ficheiro são atualizados.
- Consultar Aulas: Ao clicar no botão "Consultar Aulas", a função consultarAulas é chamada, que, por sua vez, faz o output com as informações sobre as aulas.
- Eliminar Aulas: Ao clicar no botão "Eliminar Aulas", o utilizador é solicitado a inserir o nome da aula que deseja remover. A aula correspondente é removida do ficheiro e do horário.

Considerações Adicionais

- A entrada de dados é feita através da consola (Scanner).
- Algumas operações, como adicionar e remover aulas, envolvem interação com o utilizador mediante a consola.
- As operações de adição e remoção de aulas são refletidas tanto no horário quanto em operações relacionadas a arquivos, como inserirAula e eliminarAula do objeto Ficheiro.

Estas são as principais funcionalidades do código, incluindo a interação com o utilizador, a manipulação do horário e as operações relacionadas aos arquivos.

Ficheiro()

Esta parte do código implementa a classe Ficheiro, que se responsabiliza com operações de entrada e saída relacionadas às aulas.

Estas são as principais características e métodos desta classe:

Constante

- FILE_NAME: Uma constante que representa o nome do arquivo usado para armazenar informações sobre as aulas.

Método inserirAula()

- Adiciona uma nova aula ao arquivo.
- Lê todas as aulas existentes no arquivo, adiciona a nova aula à lista e de seguida reescreve o arquivo com a lista atualizada.
- Utiliza ObjectOutputStream para escrever objetos no arquivo.

Método consultarAula()

- Lê e apresenta as aulas do arquivo.
- Utiliza ObjectInputStream para ler os objetos do arquivo.
- Mostra as informações das aulas, incluindo o nome, o treinador e os participantes.

Método eliminarAula()

- Remove uma aula do arquivo com base no nome desta.
- Lê todas as aulas do arquivo, cria uma nova lista sem a aula ser removida e reescreve o arquivo com a nova lista.
- Demonstra uma mensagem a indicar se a aula foi removida com sucesso.

Método verAula()

- Lê e devolve todas as aulas do arquivo e utiliza ObjectInputStream para ler os objetos do arquivo e adiciona-os a uma lista.

Manipulação de Exceções

- Trata exceções como FileNotFoundException, IOException, ClassNotFoundException e EOFException que podem ocorrer durante operações de I/O.

A classe Ficheiro é responsável por persistir e recuperar informações sobre as aulas em um arquivo. Ela oferece métodos para inserir, consultar e eliminar aulas, além de recuperar a lista completa destas. As operações de entrada/saída são tratadas de forma desenvolvida, lidando assim com várias exceções que podem surgir durante a execução dessas operações.

Horario()

Esta parte do código implementa a classe Horario, responsável por gerenciar o horário semanal de aulas.

Aqui estão as principais características e métodos da classe:

Atributo

- `horarioSemanal`: Um desenho que mostra os dias e os horários das aulas durante a semana, em que cada dia está associado a um conjunto de horários de aulas.

Construtor

- Inicializa o `horarioSemanal` criando desta forma mapas para cada dia da semana e horários desde as 8:00 até às 22:00, estando inicialmente sem aulas.

Método `adicionarAula`

- Adiciona uma aula ao horário em um dia e uma hora específica.
- Verifica se o dia e a hora são válidos antes de adicionar a aula.

Método `removerAula`

- Remove uma aula ao horário em um dia e uma hora específica.
- Verifica se o dia e a hora são válidos antes de remover a aula.

Método `imprimirHorario`

- Gera e imprime visualmente o horário semanal das aulas.
- Cria uma matriz representando a tabela do horário.
- Percorre o horário e preenche a matriz com os nomes das aulas ou uma string vazia se não houver aula.
- Imprime a tabela formatada com os dias da semana, as horas e os nomes das aulas.

A classe `Horario` é responsável por organizar e exibir de maneira estruturada as aulas ao longo da semana. Os métodos oferecem funcionalidades para adicionar e remover aulas, além de imprimir o horário visualmente. Este horário é um importante elemento para o sistema de gerenciamento das aulas, proporcionando assim uma visão clara das atividades programadas para cada dia e hora.

Aula()

Atributos

- hora (da aula), dia (da aula), trainer (da aula), nome (da aula) e participantes (lista de alunos que participam da aula).

Construtor

- O construtor recebe uma hora, um dia, um treinador, um nome e uma lista de participantes.
- Inicializa o `ArrayList` dos participantes.

Métodos Acessores

- `getHora()`: Retorna a hora da aula.
- `getDia()`: Retorna o dia da aula.
- `getNome()`: Retorna o nome da aula.
- `getTrainer()`: Retorna o treinador associado à aula.
- `setParticipantes (ArrayList<Aluno> participantes)`: Define a lista de participantes da aula.

Métodos de Exibição

- `mostrarTrainer()`: Constitui uma representação em formato de string com informações sobre o treinador associado à aula, incluindo nome, data de nascimento, sexo, especialidade e anos de experiência.
- `mostrarParticipantes()`: Gera uma representação em formato de string com informações sobre os participantes da aula, incluindo nome, data de nascimento, sexo, nível de condicionamento e objetivo.

A classe `Aula` implementa a interface `Serializable`, demonstrando desta forma que as suas instâncias podem ser serializadas para armazenamento ou transmissão. Esta classe engloba as informações relacionadas a uma aula, permitindo a exibição detalhada do treinador e dos participantes.

Pessoa()

Esta parte do código define a classe Pessoa, que implementa a interface Serializable. Aqui está um resumo das principais características e métodos da classe:

Atributos

- nome (da pessoa), dob (data de nascimento) e sexo (da pessoa).

Construtores

- Pessoa(): Um construtor padrão que inicializa os atributos com valores predefinidos (espaços em branco).
- Pessoa(String nome, String dob, String sexo): Um construtor que recebe informações específicas da pessoa e as atribui aos atributos correspondentes.

Métodos Acessores

- getNome(): Retorna o nome da pessoa.
- getDob(): Retorna a data de nascimento da pessoa.
- getSexo(): Retorna o sexo da pessoa.

Método toString()

- Subscrive o método toString(), que retorna o nome da pessoa.

A classe Pessoa é uma classe base para as classes Aluno e Trainer, fornecendo informações básicas compartilhadas entre ambas. Os métodos acessores permitem recuperar as informações específicas de uma pessoa, como nome, data de nascimento e sexo. O método toString() fornece uma representação em formato de string do nome da pessoa.

Aluno()

Esta parte do código define a classe Aluno, que recebe da classe Pessoa e implementa a interface Serializable.

Estas são as principais características e métodos desta classe:

Atributos

- nivelDeCondicionamento (exemplo: iniciante, intermediário, avançado) e objetivo (exemplo: perda de peso, ganho de massa muscular, condicionamento físico).

Construtores

- Aluno(): Um construtor predefinido que inicializa os atributos com valores certos (que pode ser ajustado conforme o necessário).
- Aluno(String nome, String dob, String sexo, String nivelDeCondicionamento, String objetivo): Um construtor que recebe informações específicas do aluno, chamando o construtor da classe pai Pessoa para inicializar informações básicas e definindo o nível de condicionamento e objetivo.

Métodos Acessores

- getNivelDeCondicionamento(): Retorna o nível do condicionamento físico do aluno.
- getObjetivo(): Retorna o objetivo do aluno na prática dos exercícios.

Método toString()

- Subscrive o método toString() da classe Pessoa, incluindo informações sobre o nível de condicionamento e objetivo do aluno.

A classe Aluno é projetada para representar um aluno no sistema de gerenciamento das aulas, fornecendo informações sobre a sua identidade, o nível de condicionamento físico e o objetivo do treino.

Trainer()

Esta parte do código define a classe Trainer, que herda da classe Pessoa.

Aqui estão as principais características e métodos da classe:

Atributos

- especialidade (exemplo: yoga, musculação, pilates, etc) e anosDeExperiencia (do treinador na sua especialidade)

Construtor

- Trainer(String nome, String dob, String sexo, String especialidade, int anosDeExperiencia): Um construtor que recebe informações específicas do

treinador, chamando o construtor da classe pai Pessoa para inicializar informações básicas e definindo a especialidade e os anos de experiência.

Métodos Acessores

- `getEspecialidade()`: Retorna a especialidade do treinador.
- `getAnosDeExperiencia()`: Devolve o número de anos de experiência que o treinador tem na sua especialidade.

Método toString()

- Subscrive o método `toString()` da classe Pessoa, incluindo informações sobre a especialidade e os anos de experiência do treinador.

Esta classe Trainer tem como objetivo representar um treinador no sistema de gerenciamento de aulas, fornecendo informações sobre a sua identidade, a sua especialidade e a sua experiência. Ela herda as características básicas de uma pessoa da classe Pessoa. O método `toString()` fornece uma representação em formato de string das informações do treinador, incluindo detalhes específicos sobre a sua especialidade e a sua experiência.